ChatGPT said:

ServiceIQ — Problem Statement & RAG Solution Framework

AI-Powered Customer Knowledge & Service Intake Assistant | Version 1.0

1. Executive Summary

ServiceIQ is an AI-powered customer engagement solution designed for small and medium-sized HVAC service businesses. It uses Retrieval-Augmented Generation (RAG) to provide customers with accurate, contextually relevant, source-grounded answers through a chatbot embedded on the company's website.

The initial solution focuses on answering customer inquiries using approved company knowledge, with a target of **≥95% answer faithfulness**, measured through a repeatable evaluation framework.

The long-term vision is to expand ServiceIQ from a knowledge assistant into an AI-powered customer intake and service scheduling platform.

2. Business Problem Statement

Small and medium-sized HVAC service companies rely on fragmented information sources—including websites, service catalogs, equipment manuals, warranty documents, pricing sheets, and FAQs—to respond to customer inquiries.

Customers frequently contact these businesses with repetitive questions about services, pricing, equipment compatibility, warranties, maintenance, and appointment requests.

Traditional website FAQs and rule-based chatbots cannot reliably interpret natural-language questions or synthesize information from multiple documents. Generic LLM-based chatbots may produce inaccurate answers, invent prices or policies, and provide information inconsistent with company-approved sources.

As a result, businesses may experience increased administrative workload, inconsistent customer responses, missed service opportunities, and reduced customer confidence.

Problem to solve

> **How can an HVAC service company provide customers with accurate, trustworthy, 24/7 answers using its existing business knowledge, while reducing repetitive administrative work and converting customer inquiries into actionable service requests?**

Desired business outcomes

Business Objective	Expected Outcome	Proposed KPI
Improve answer reliability	Responses grounded in approved company knowledge	≥95% faithfulness
Reduce repetitive inquiries	Resolve common questions without employee intervention	≥40% self-service resolution, pilot hypothesis
Improve customer experience	Provide answers through a 24/7 website chatbot	≥85% positive feedback, pilot hypothesis
Increase service opportunities	Convert qualified inquiries into service requests	Track inquiry-to-request conversion
Minimize misinformation	Avoid unsupported pricing, warranty, and policy statements	≥95% citation accuracy

Maintain current knowledge	Automatically synchronize approved documents	Meet defined freshness SLAs

Business KPIs are initial hypotheses to validate during a pilot, not established performance benchmarks.

3. Solution Vision and Scope

ServiceIQ

Ask. Understand. Resolve. Connect.

MVP: RAG chatbot

MVP — Knowledge Assistant

Customers ask questions in natural language, and ServiceIQ retrieves relevant company documents and generates concise answers with verifiable citations.

Phase 2 — AI Service Intake

ServiceIQ identifies customer intent, collects equipment and issue details, and creates structured service requests with human handoff.

Phase 3 — Agentic Service Operations

ServiceIQ integrates with CRM, scheduling, work-order, and customer communication systems to support end-to-end service workflows.

MVP functional scope

In Scope	Out of Scope for MVP
Website chatbot	Autonomous appointment booking
Approved knowledge ingestion	Live CRM/ERP integrations
Semantic knowledge search	Equipment diagnosis or repair instructions beyond approved safe guidance
Grounded response generation	Automated pricing quotations
Source citations	Payment processing
Unanswerable-question handling	Autonomous work-order dispatch
Automated RAG evaluations	Voice/phone agents
Basic conversation logging	Fully autonomous multi-agent orchestration

4. RAG Design Framework

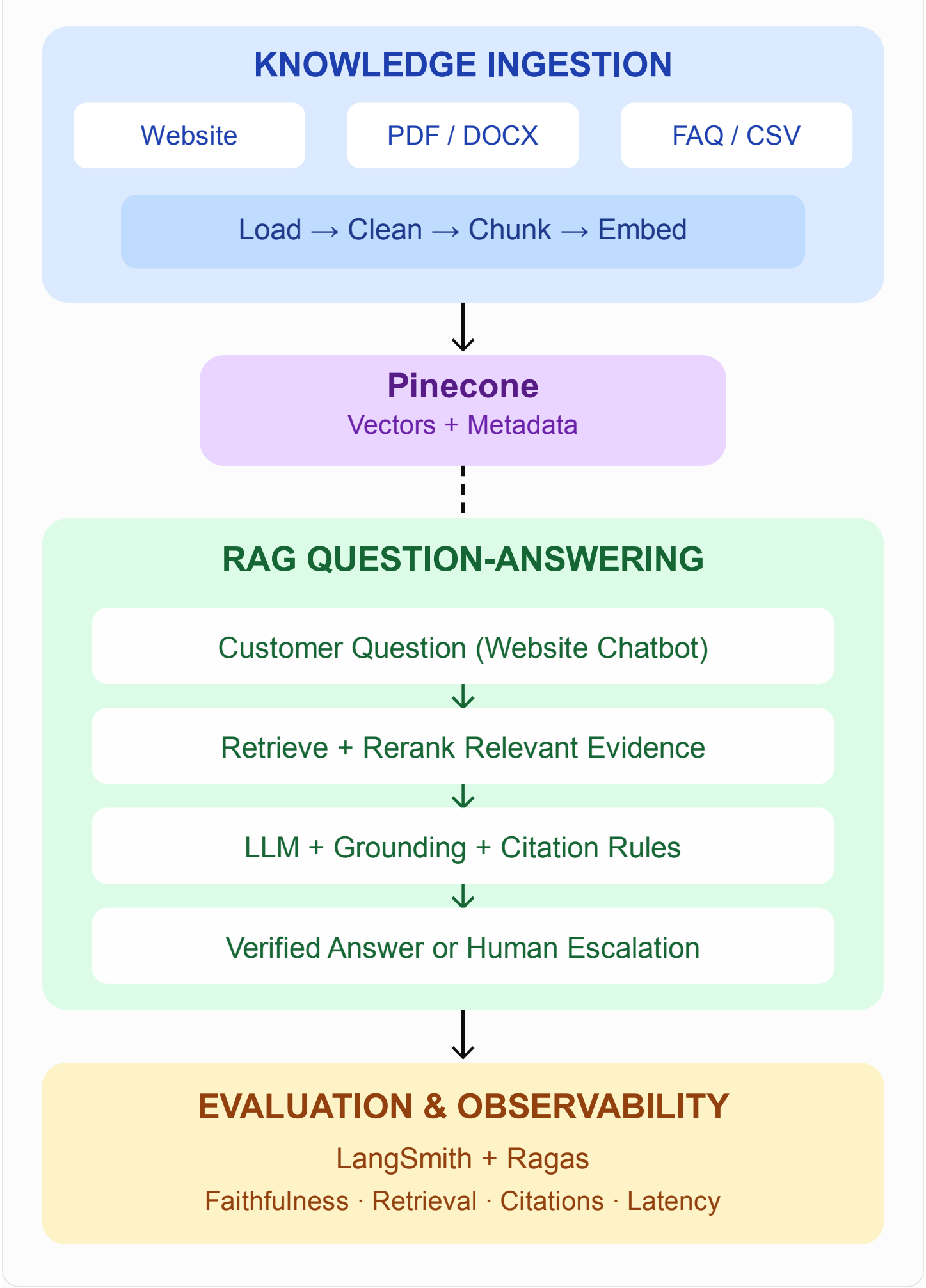
The following defines the initial architectural decisions for ServiceIQ.

Field	ServiceIQ Framework
Use case	HVAC customers ask service, pricing, warranty, maintenance, and coverage questions through a website chatbot. The system answers using authorized company documents and guides customers toward human assistance or service intake where appropriate.
Corpus	Initially 50–100 approved English documents, including FAQs, service catalogs, price lists, warranty policies, maintenance guides, and manufacturer documentation in HTML, PDF,

	DOCX, and CSV formats. The company knowledge administrator owns content approval and authoritative source designation.
Ingestion + cleaning	Use LangChain document loaders and website content extraction to ingest documents. Remove HTML navigation, duplicate content, irrelevant boilerplate, and obsolete versions while preserving headings, pricing tables, warranty conditions, document provenance, and effective dates.
Ingestion + freshness	Perform initial bulk ingestion, daily website synchronization, and event-triggered reindexing following document approval. Target 15-minute freshness for pricing, policies, and warranty changes and 24 hours for general knowledge, with version tracking and update-failure alerts.
Chunking + embedding	Start with structure-aware chunks of approximately 500 tokens and 75-token overlap, preserving complete FAQs and logically related policy clauses. Use OpenAI `text-embedding-3-small` and evaluate chunk sizes against retrieval quality and cost.
Retrieve	Use Pinecone dense vector retrieval with metadata filtering and an initial `top_k=10`. Evaluate adding hybrid keyword/vector search for model numbers, service codes, and exact terminology.
Reranking	Rerank the retrieved candidates by relevance and select approximately five evidence chunks for context assembly. Measure incremental retrieval improvement versus latency and cost.
Generation	Generate concise, customer-friendly responses using an OpenAI model orchestrated through LangChain. Require supporting citations, prohibit unsupported factual assertions, and return an explicit fallback when authoritative information is missing.
Grounding	Only approved and current sources may support answers; conflicting sources must be resolved by document authority and effective date or escalated. Treat retrieved content as untrusted instructions, even when its facts are approved.
Metadata	Track `tenant_id`, `document_id`, `source`, `document_type`, `approval_status`, `version`, `effective_date`, `expiry_date`, and `access_scope`. Enforce tenant and authorization isolation before retrieval.
Evaluations	Use LangSmith and Ragas for regression testing, retrieval analysis, faithfulness evaluation, and application tracing. Maintain at least 100 representative test questions, reference answers, and known relevant source documents.
Quality targets	Target ≥95% faithfulness, ≥90% Recall@5, and ≥95% citation accuracy on the evaluation dataset. Separately test price accuracy, warranty correctness, unsupported-answer handling, security, response latency, and cost.
Observability	Capture traces, retrieval results, answer citations, model usage, latency, errors, and anonymized feedback through LangSmith and application monitoring. Flag problematic conversations for investigation.
Human escalation	Escalate questions when evidence is insufficient, sources conflict, the user requests a binding quote, or the request requires a licensed technician or company representative. Capture the conversation context for a smooth handoff.

5. High-Level Solution Architecture

ServiceIQ — Logical Architecture


KNOWLEDGE INGESTION
Website
PDF / DOCX
FAQ / CSV
Load → Clean → Chunk → Embed
Pinecone
Vectors + Metadata
RAG QUESTION-ANSWERING
Customer Question (Website Chatbot)
Retrieve + Rerank Relevant Evidence
LLM + Grounding + Citation Rules
Verified Answer or Human Escalation
EVALUATION & OBSERVABILITY
LangSmith + Ragas
Faithfulness · Retrieval · Citations · Latency

The ingestion and question-answering processes operate independently. Documents are indexed when updated; live customer questions are answered through retrieval against the current approved knowledge base.

6. Evaluation and Acceptance Framework

Our primary engineering requirement is **at least 95% faithfulness**, meaning factual claims in generated answers must be supported by retrieved evidence. This follows the Ragas faithfulness metric definition.

GitHub

Metric	MVP Target	Validation Method
Faithfulness	≥95%	Ragas claim-grounding evaluation
Retrieval Recall@5	≥90%	Compare retrieved document IDs with labeled relevant sources
Answer correctness	≥90%	Compare against human-approved reference answers
Citation accuracy	≥95%	Verify that cited passages support factual claims
Unsupported-answer handling	≥95%	Test queries with missing or insufficient evidence
Critical pricing/policy errors	0 in acceptance suite	Human-verified regression tests
P95 response latency	≤5 seconds	End-to-end application timing

These are acceptance targets to test, not expected or guaranteed outcomes. Automated evaluator results should be calibrated with human review, especially for pricing and warranty scenarios.

Example evaluation scenarios

Customer Question	Expected Behavior
Do you repair Carrier AC units?	Retrieve approved service catalog and answer with citation
How much is a diagnostic visit?	Return current approved diagnostic pricing
Is my compressor under warranty?	Explain warranty conditions without asserting coverage before verification
Do you have an appointment tomorrow?	Explain that live availability cannot be confirmed without scheduling integration
What should I do if I smell gas?	Prioritize emergency safety guidance and escalation, not ordinary troubleshooting
What does another HVAC company charge?	Acknowledge missing authoritative information rather than invent competitor prices

7. Implementation Plan

Sprint 1 — Foundational RAG

Build document ingestion, chunking, embeddings, Pinecone retrieval, grounded generation, and the initial website chatbot.

Deliverable: Working RAG MVP

Sprint 2 — Evals and Retrieval Optimization

Build the 100-question evaluation set, establish a baseline, tune retrieval, introduce reranking where justified, and implement automated regression testing.

Deliverable: Measurable RAG quality

Sprint 3 — Production Hardening

Implement access controls, source versioning, monitoring, feedback capture, failure handling, and deployment automation.

Deliverable: Pilot-ready knowledge assistant

Sprint 4 — Agentic Service Intake

Introduce LangGraph-based state management, customer qualification, structured service requests, and human handoff without coupling those workflows to the core RAG engine.

Deliverable: Service intake prototype

8. Definition of Done — RAG MVP

ServiceIQ's RAG MVP will be considered complete when the website chatbot can answer representative HVAC customer questions using approved company knowledge, display traceable source citations, safely abstain when evidence is insufficient, and meet the agreed evaluation thresholds on a held-out dataset.

The solution must also support knowledge updates, basic error handling, traceability, and reproducible deployment. The initial portfolio demonstration should show both successful customer interactions and documented evaluation results, including the improvements made to reach the target.

Recommended technical stack: Python, FastAPI, LangChain, Pinecone, OpenAI embeddings and LLM, Streamlit for the initial chatbot interface, LangSmith, Ragas, Docker, and GitHub.

This framework establishes a measurable starting point for ServiceIQ while keeping the core RAG capability reusable for future field technician copilots, service intake agents, and industry-specific applications.